

Construct Uniform Parity Array - II

© Authored by: Arpit Raj, LNMIIT Jaipur | Date: September 4, 2026

1. Problem Understanding

Given **nums1** array, need to create **nums2** array such that all nums in **nums2** are either all odd or all even.

2. Allowed Operations

$\text{nums2}[i] = \text{nums1}[i]$

$\text{nums2}[i] = \text{nums1}[i] - \text{nums1}[j]$

Such that $j \neq i$ and $\text{nums1}[i] - \text{nums1}[j] \geq 1$

3. Core Logic & Examples

Basically, consider the array $[1, 4, 7]$.

- To make all odd $\rightarrow \text{even} - \text{odd} = \text{odd}$
- To make all even $\rightarrow \text{odd} - \text{odd} = \text{even}$

So applying this to our example:

- $4 - 1 = 3 \rightarrow$ **Allowed**
- $4 - 7 = -3 \rightarrow$ **Not allowed**

Hence, we can form the array $[1, 3, 7]$, so the condition holds **True**.

4. Crucial Observation (Note)

Note $\rightarrow$ even $\rightarrow$ all odd array is possible.

But $\rightarrow$ odd $\rightarrow$ all even array is **not possible**.

Example: $[1, 4, 7]$, you can do $(7 - 1) = 6$ to get $[1, 4, 6]$.

But how to make 1 into an even number? Because $1 - 7 = -6$ (which is not allowed). Hence, because of the smallest odd, we can't have an all even array.

5. Approach & Optimization

Approach: Create a separate array for even and odd.

i.e., for $[1, 4, 7] \rightarrow$ Odds: $[1, 7]$, Evens: $[4]$

Initial thought: Now using lower bound, look for a num in odd smaller than num in even.

Optimized logic: We can avoid lower bound. We can just sort the even and odd arrays.

Example: $[1, 6, 4, 7]$

- Odds: $[1, 7]$

- Evens: $[4, 6]$

Sorted the smallest, now just compare 2 numbers, i.e., 1 and 4.

Therefore, an all odd array can be made because $(4 - 1 \geq 1)$. Since $(4 - 1)$ is valid, $(6 - 1)$ is guaranteed to be $\geq (4 - 1)$ & odd.

6. C++ Code Implementation

```
class Solution {
public:
    bool uniformArray(vector<int>& nums1) {
        int n = nums1.size();

        vector<int> oddarr;
        vector<int> evarr;

        for(int num : nums1) {
            if(num % 2 == 0) {
                evarr.push_back(num);
            }
            else {
                oddarr.push_back(num);
            }
        }

        sort(evarr.begin(), evarr.end());
        sort(oddarr.begin(), oddarr.end());

        if(oddarr.empty()) return true;
        if(evarr.empty()) return true;

        // saare odd even case
        if(evarr[0] > oddarr[0]) return true;
        else return false;
    }
};
```